40 prompts para programar con IA (español)

Cómo usarlos: reemplaza lo que va entre [corchetes]. Pega el código real; mientras más contexto des, mejor sale. Revisa y prueba todo lo que te devuelva la IA.

Entender código

1. Explica este código línea por línea como si fuera mi primer día programando: [código]
2. Dime qué hace esta función, qué entradas espera, qué devuelve y qué casos borde puede romperla: [código]
3. Dibuja en texto el flujo de ejecución de este programa, con las llamadas en orden: [código]
4. Resume este repositorio: qué hace, carpetas importantes y por dónde empezar a leer. Estructura: [árbol de archivos]
5. ¿Qué dependencias de este archivo podrían ser un riesgo (obsoletas, inseguras, innecesarias)? [package.json / requirements.txt]

Depurar

6. Este error aparece al hacer [acción]. Dame las 3 causas más probables ordenadas por probabilidad y cómo comprobar cada una: [error + código]
7. Funciona en mi equipo pero falla en [entorno]. Lista las diferencias típicas que debo revisar: [detalles]
8. Añade logs mínimos a este código para localizar dónde se rompe, sin cambiar su lógica: [código]
9. Dame un caso mínimo reproducible a partir de este código y error: [código + error]
10. Revisa este código buscando bugs de lógica, no de estilo. Para cada uno: línea, por qué falla y arreglo: [código]

Escribir código

11. Escribe una función en [lenguaje] que [objetivo]. Entradas: [x]. Salidas: [y]. Maneja [casos borde]. Sin librerías externas.
12. Genera un script de [bash/python] que [tarea] y falle con mensaje claro si falta algún dato.
13. Crea un CLI en [lenguaje] con argumentos [lista] y ayuda `--help`.
14. Escribe una API REST mínima en [framework] con los endpoints [lista], validación de entrada y respuestas JSON consistentes.
15. Convierte este código de [lenguaje A] a [lenguaje B] conservando el comportamiento y explica las diferencias: [código]
16. Escribe una expresión regular que [objetivo]. Dame 5 ejemplos que coinciden y 5 que no.
17. Escribe la consulta SQL para [objetivo] con estas tablas: [esquema]. Explica el índice que ayudaría.

Mejorar código

18. Refactoriza esto para que sea más legible sin cambiar su comportamiento. Explica cada cambio: [código]
19. Encuentra qué parte de este código es lenta y propón una mejora, indicando la complejidad antes y después: [código]
20. Divide esta función larga en funciones pequeñas con nombres claros: [código]
21. Renombra variables y funciones confusas; dame una tabla antiguo → nuevo con la razón: [código]
22. Quita duplicación en este código y dime si la abstracción realmente vale la pena: [código]

Pruebas

23. Escribe pruebas unitarias con [framework] para esta función, incluyendo casos borde y errores esperados: [código]
24. ¿Qué casos de prueba me faltan? Lista solo los que detectarían bugs reales: [código + pruebas]
25. Haz una prueba de integración para este endpoint con datos de ejemplo: [código]

Documentación y Git

26. Escribe un README en español para este proyecto: qué es, instalación, uso con ejemplos y licencia: [descripción/código]
27. Añade docstrings/comentarios solo donde el *porqué* no sea obvio: [código]
28. Redacta un mensaje de commit (asunto de máx. 50 caracteres + cuerpo) para este diff: [diff]
29. Escribe la descripción de un pull request: qué cambia, por qué y cómo probarlo: [diff]
30. Resume estos cambios para un changelog orientado al usuario: [commits]

Seguridad y calidad

31. Revisa este código en busca de inyección, XSS, secretos expuestos y validación insuficiente. Para cada hallazgo, impacto y arreglo: [código]
32. ¿Qué datos de usuario maneja este código y cómo deberían protegerse? [código]
33. Haz una checklist de revisión de código para un PR de [tipo de cambio] en [stack].

Aprendizaje y carrera

34. Explícame [concepto] con una analogía cotidiana y un ejemplo mínimo en [lenguaje].
35. Dame un plan de 4 semanas, 1 hora diaria, para aprender [tecnología] haciendo un proyecto real.
36. Hazme 10 preguntas de entrevista técnica de [tema], con la respuesta esperada y errores comunes.

37. Dame 5 ideas de proyecto de portafolio con [stack], ordenadas de fácil a difícil, con qué demuestra cada una.

Producto y planeación

38. Convierte esta idea en historias de usuario con criterios de aceptación: [idea]
39. Propón la arquitectura mínima viable para [producto] con [restricciones]; qué dejarías fuera de la primera versión.
40. Compara [opción A] vs [opción B] para [caso de uso] en una tabla: costo, complejidad, riesgos. Termina con una recomendación.

Reglas de oro

- Pide siempre: “explica tus supuestos y dime qué no pudiste verificar”.
- Cambios pequeños y probados: un prompt, un cambio, una prueba.
- Nunca pegues contraseñas, tokens ni datos personales reales.